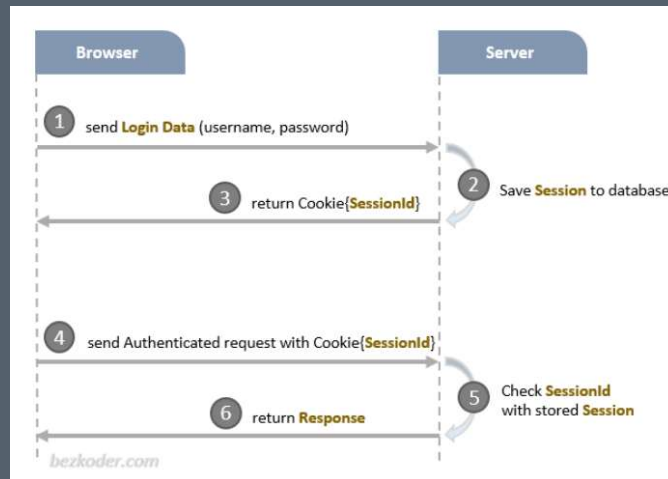


AUTH UND STORAGE

JWT VS. SESSION-BASED AUTHENTICATION

SESSION-BASED AUTHENTICATION

- Bei Login wird SessionId generiert und in einem Browser Cookie abgelegt
- Der Cookie wird dann bei allen HTTP-Requests mitgeschickt
- Cookies zwingend erforderlich, daher nicht für Native Apps geeignet



© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

200

JWT (JSON WEB TOKEN)

- Bei Login wird ein JWT generiert und an den Client geschickt
- Der JWT wird anschließend bei den HTTP-Requests mitgeschickt
- Kein Cookie erforderlich, damit auch für Native Apps geeignet

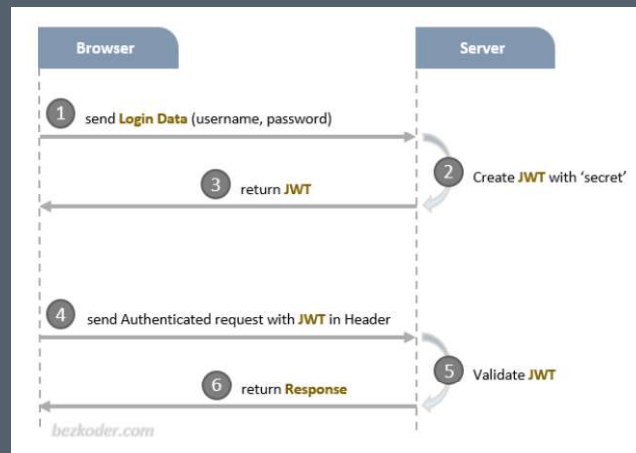
© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

201

- Beispiel JWT: eyJhbG9jaXZpdF(...)yNzAl9.bVig(...)9YKzQ
- JWT besteht aus header.payload.signatur:
 - Header: Gibt an wie der JWT generiert wird
 - Payload: Daten die im JWT gespeichert werden (in unserem Fall User-ID und Expiration Time des Tokens)
 - Signatur: Signatur des JWT um Manipulation zu verhindern

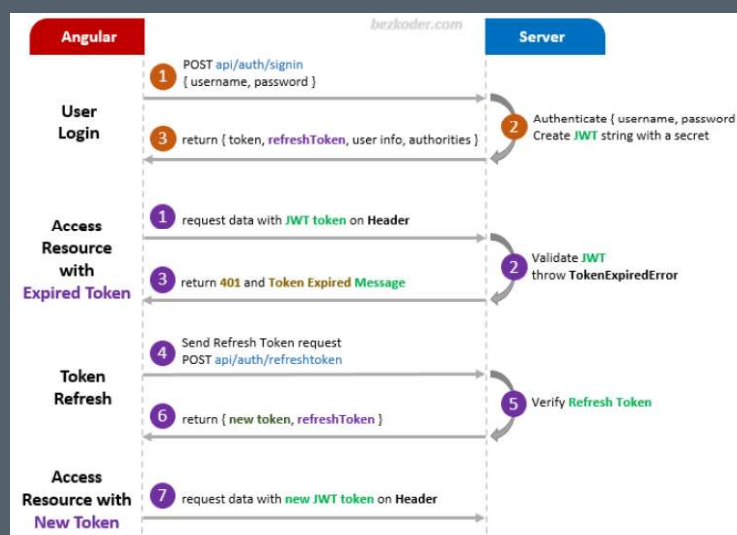
© 2025 S&N Invent - Dominik Böcker, Dimitrij Ron

202



JWT REFRESH TOKEN

- Zusätzlich zum JWT (AccessToken) kann auch ein RefreshToken verwendet werden
- Der RefreshToken wird beim Login erstellt und zusammen mit dem AccessToken ans Frontend geschickt
- Der RefreshToken hat dabei eine längere Gültigkeit als der AccessToken
- Wenn der AccessToken abgelaufen ist, kann mit dem RefreshToken ein neuer AccessToken angefordert werden (kein erneuter Login notwendig)



SESSIONSTORAGE VS. LOCALSTORAGE

SESSIONSTORAGE

- Abgelegte Daten sind nur für die Session (aktueller Tab) gültig
- SessionStorage wird geleert wenn (Browser-)Tab geschlossen wird
- Jeder Tab hat seinen eigenen SessionStorage und kann nicht auf den SessionStorage anderer Tabs zugreifen
- Zugreifbar über Objekt "sessionStorage"

LOCALSTORAGE

- Abgelegte Daten sind für alle Sessions (Tabs) mit gleicher Origin zugänglich
- LocalStorage bleibt bestehen wenn Tab/Browser geschlossen werden
- Jeder Tab kann auf die im LocalStorage abgelegten Daten zugreifen, solange die selbe Origin vorliegt
- Zugreifbar über Objekt "localStorage"

APIS

- Local- und SessionStorage verfügen über die gleiche API
- Daten werden dabei als Key/Value Paare abgelegt

```
sessionStorage.setItem("key", "value");
```

- Über den Key können dann die Daten gelesen werden

```
let lastname = sessionStorage.getItem("key");
```

- Über den Key können Daten entfernt werden

```
sessionStorage.removeItem("key");
```

- Es können aber auch alle Daten entfernt werden

```
sessionStorage.clear();
```

- Daten können nur in Form von Strings abgelegt werden
- Wenn wir komplexe Daten im Session- oder LocalStorage ablegen wollen, müssen wir diese vorher in einen JSON-String konvertieren

```
sessionStorage.setItem(USER_KEY, JSON.stringify(user));
```

- Beim Laden können wir diese dann wieder zurück konvertieren

```
const user = window.sessionStorage.getItem(USER_KEY);  
if (user) {  
  return JSON.parse(user);  
}
```